CLAIMSHIELD AI

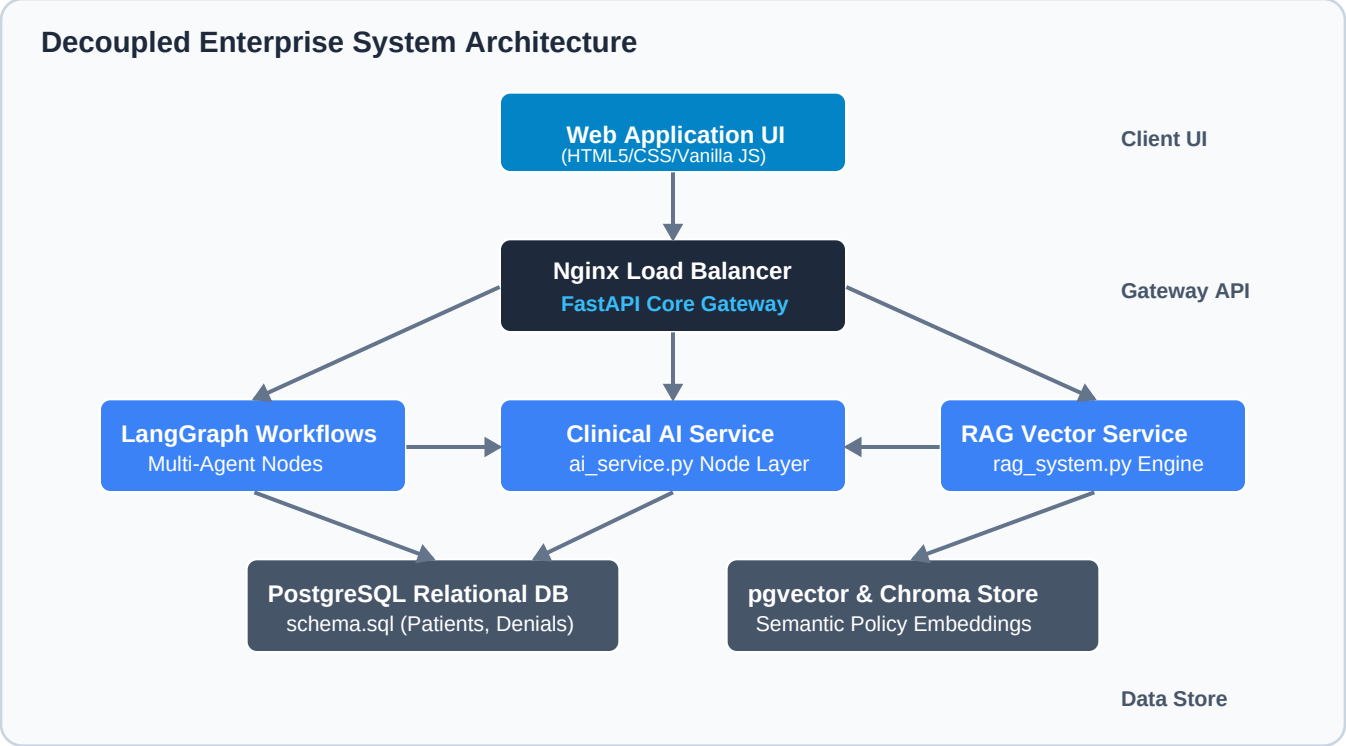
SYSTEM ARCHITECTURE BLUEPRINTS

Project Component: Architecture Diagram & System Design Spec
Target Domain: Healthcare Revenue Cycle Management (RCM) & AI Automation
Technical Environment: FastAPI Gateway, pgvector Relational/Semantic DB, LangGraph Multi-Agent Workflows, and Local Model Inference
Author / Team: Solo Builder Submission
Date: June 2026

Architectural Overview:
The ClaimShield AI platform utilizes a decoupled, high-performance microservices architecture. The core application separates user interaction (Client Tier), requests routing and stateful scheduling (API and Orchestration Tier), heavy analytics operations (Service Tier), and relational/vector storage (Storage Tier). Through asynchronous queuing and semantic vector search, ClaimShield AI coordinates specialized AI agents safely, maintaining high throughput and HIPAA privacy compliance. This document presents the detailed architectural design and flow patterns of the platform.

1. Decoupled System Architecture

ClaimShield AI is structured into four distinct layers. This separation isolates heavy model inference and document indexing loads from user interfaces, maintaining system responsiveness even during high-volume batch ingestions. The diagram below details the communication paths between these tiers.

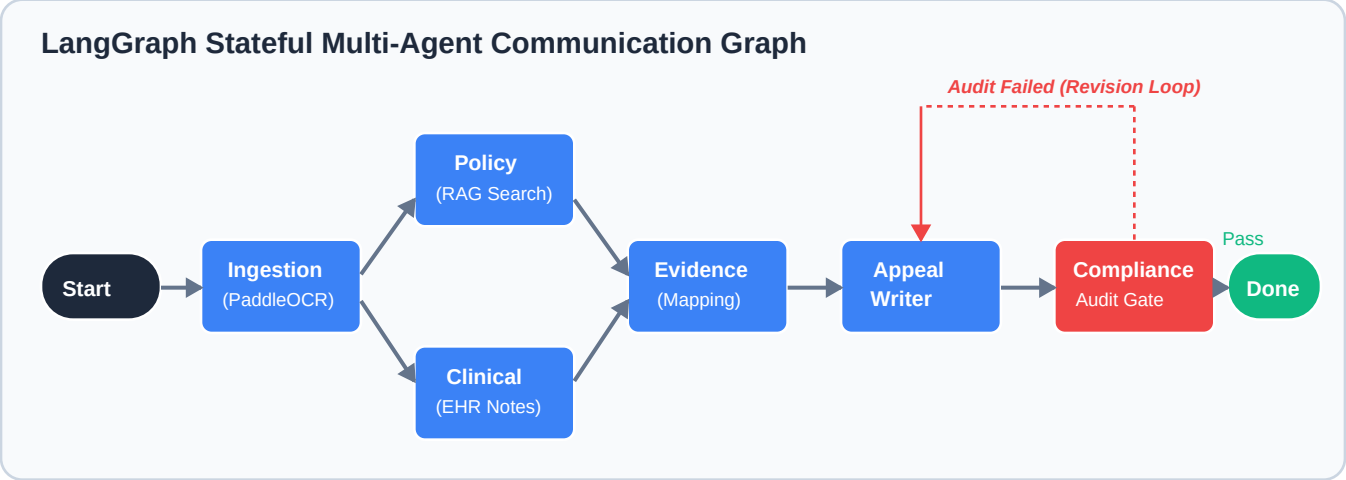


System Component Tiers:

- **Client Tier (Web UI):** Single-Page web application built in HTML5/CSS/Vanilla JS, incorporating Google Single Sign-On and a strict 5-minute inactivity session lock modal for HIPAA compliance.
- **API & Orchestration Tier:** Nginx reverse proxy routing traffic securely to a FastAPI Gateway. The gateway coordinates database states, caches session/policies, and executes stateful multi-agent workflows using LangGraph.
- **Clinical & Search Services:** Specialized Python services. The RAG Engine (rag_system.py) maps policy embeddings, and the AI Service (ai_service.py) manages prompt formats and interacts with local LLM hosts.
- **Storage Tier:** PostgreSQL stores transactional billing and patient data. A pgvector extension manages coverage policy embeddings, while Redis caches frequent policy queries and session tokens.

2. LangGraph Stateful Multi-Agent Communication Graph

Rather than relying on single-prompt operations, the core reasoning layer of ClaimShield AI is compiled as a stateful Directed Acyclic Graph (DAG) using **LangGraph**. The graph coordinates 7 specialized agents, representing distinct workflow nodes. The Compliance Agent acts as a strict audit gateway, directing the appeals back to the Appeal Writer if verification rules fail.



Specialized Agent Node Responsibilities:

1. **Ingestion Agent:** Scans uploaded PDF denial notices using PaddleOCR/Tesseract. Parses unstructured layouts for metadata (Claim Number, Payer, CARC code, Denial reasons).
2. **Policy Agent:** Receives claim metadata and queries the pgvector database for the exact insurance carrier guidelines, returning coverage clauses.
3. **Clinical Agent:** Parses patient clinical charts and progress notes to extract symptoms, diagnoses, failed treatments, and lab dates.
4. **Evidence Agent:** Cross-references the extracted clinical indicators against the retrieved payer rules, establishing a comparison matrix of satisfied requirements and missing gaps.
5. **Appeal Writer:** Merges the comparison evidence matrix into customized medical necessity templates, compiling a comprehensive appeal letter.
6. **Compliance Agent (Audit Gate):** Analyzes the letter for HIPAA compliance (verifying that no unnecessary PHI is leaked), codes matching, and CARC alignment. If any check fails, it triggers the revision loop.
7. **Follow Up Agent:** Finalizes the appeal record, registers follow-up tasks in the database, and schedules calendar/deadline reminders.